Why Overlay Cannot Be Used

Risks of the Raspberry Pi read-only overlay for the RFID broker, and why it does not fit

Short answer

- The overlay protects the SD card by sending **every write to RAM**. Everything written since boot is **lost at reboot or power cut**.
- The broker must keep the **latest settings and RFID data** after any reboot or power cut. The overlay makes that impossible without extra storage.
- Therefore overlay cannot be used on its own for this system. Hardening is used instead.

1. How the overlay works

The root filesystem on the SD card is mounted read-only (lower layer). A temporary layer in RAM (upper layer) receives all changes. The application sees both layers as one filesystem. The SD card is never written, so a power cut cannot corrupt it - but the RAM layer is empty again at the next boot, so the system returns to exactly what is stored on the SD card (the image).

2. What happens to the broker's files

File	What the broker uses it for	Result with overlay
config.json	Device settings; the Pi reboots automatically after a change	New settings are lost at that very reboot - the original problem
conf.d/bridge.conf	MQTT bridge identity and broker address	Reverts to the image version (the master unit's cabinet name / address)
rfid2.json	Latest scan result	Reverts to old data
rfid1.json	Previous scan, compared with the new scan to compute add / remove	Stale snapshot, so add / remove can be reported wrongly
status.json	Status, session ID, light state	Session and light state reset
logs/	Diagnostic logs	Lost at reboot; can fill RAM while running

3. Risks of using the overlay

Risk	Likelihood	Consequence
Settings and scan data revert at every reboot	Certain	Changes are lost; wrong add / remove; the goal of keeping the latest data is not met.
Reboot loop caused by ipservice.py on cloned units	Likely on clones (derived from the code, not yet tested on a Pi)	bridge.conf reverts to the image version at every boot; ipservice.py rewrites it (in RAM) and reboots whenever the cabinet letter differs from the image; the rewrite is lost and the cycle repeats.
Hostname changes are lost	Certain	hostnamectl is reverted at reboot; ipservice.py must redo it at every boot.
RAM fills with logs	Medium (logger on)	Up to about 8 MB per logger file set; the system can slow down or stall.
Maintenance and remote updates	Always	Disable overlay, reboot, change, enable, reboot. Hard without physical access.
Keeping some files persistent by re-opening the SD card for writing	If attempted	Brings back write risk on the OS partition; fragile and untested; a mistake can stop the Pi booting.

4. Example: the reboot loop on a cloned unit

- The master Pi has IP .101, so the image holds bridge.conf with the cabinet name ending in **A**.
- The image is cloned to a cabinet whose IP is .103 (letter **C**).
- At boot, bridge.conf says A. ipservice.py sees IP .103, writes C (only into RAM) and reboots.
- RAM is cleared, bridge.conf is back to A. The state file in /var/tmp is lost too, so the same steps repeat at every boot.

Without overlay (with hardening) the rewritten bridge.conf stays on the SD card, so there is exactly one reboot when the IP or name changes.

5. Why it cannot be used here

Requirement	Overlay
Latest settings and scan data kept after reboot / power cut	Not met
Settings survive the automatic reboot after a change	Not met
Suitable for remote cabinets (no physical access)	Not suitable
Works without an extra partition or storage	Not possible - overlay needs separate writable storage to keep any data
No changes to existing code	Met, but data resets at every boot

Conclusion: the overlay would protect the SD card but defeat the purpose of the system. Use **hardening** (atomic writes, logs in RAM, fewer writes) as described in the standard installation guide. Overlay would only become suitable if a separate writable storage (for example a second partition or a USB drive) could be added for the data files.

6. Note

The reboot-loop analysis is based on reading ipservice.py and has not been tested on a Raspberry Pi with the overlay enabled. No method can fully guarantee the SD card never fails: an industrial or pSLC card and a small UPS remain the strongest additional protections.